Week 7 (2025-11-14)

This week's quizzes

- [Week 7 activity (11.14.) (https://canvas.elte.hu/courses/60916/quizzes/186590)](https://canvas.elte.hu/courses/60916/quizzes/186590)
- [Week 7 mini quiz (11.14.) (https://canvas.elte.hu/courses/60916/quizzes/186579)](https://canvas.elte.hu/courses/60916/quizzes/186579)

Modularization

Large software systems are written by many people, and they organize their work into hundreds of directories and files. To understand the problems this causes, we tried to model it with just three files and a very basic domain about pets.

We discussed that compilation has three stages: *preprocessor*, *compiler*, and *linker*. This picture is more detailed than what I actually drew, but it helps with understanding.

```
                .h files
                    |
                    ▼
                                                  libraries
 file1.c ──►┌──────────────┐  ┌──────────┐                  |
            │ preprocessor├─►│compiler  ├─► file1.o ─┐      ▼
            └──────────────┘  └──────────┘            |
            ┌──────────────┐  ┌──────────┐            |  ┌───────┐
 file2.c ──►│ preprocessor├─►│compiler  ├─► file2.o ─┼─►│linker├─► program
            └──────────────┘  └──────────┘            |  └───────┘
            ┌──────────────┐  ┌──────────┐            |
 file3.c ──►│ preprocessor├─►│compiler  ├─► file3.o ─┘
            └──────────────┘  └──────────┘
```

Each of your .c file is converted to something called an object file first, once they are ready, the linker combines them with the libraries you use and creates the final output program. Header files contain shared type definitions, and tell the compiler what functions are defined in the libraries and other .c files. While libraries contain the actual implementation of `printf`, `scanf`. Object files (`.o`) also contain implementation of the functions originating from the corresponding `.c` file.

There is a separate pipeline for each .c file at the beginning, and you can trigger the whole machinery with a simple `gcc file1.c file2.c file3.c -o program` command, where `program` is the name of the final output (by default it would be `a.out`).

We created three files `pet.h`, `pet.c`, and `main.c`; defined a struct and an enum in `pet.h`, and included it in our `main.c`:

```c
// pet.h
enum PetCategory {
    PET_CATEGORY_DOG,
    PET_CATEGORY_CAT,
    PET_CATEGORY_BIRD,
    PET_CATEGORY_FISH,
};

struct Pet {
    char name[33];
    enum PetCategory category;
};

// main.c
#include <stdio.h>
```

```
#include "pet.h"

int main(void){
   struct Pet p = {.name="Fred", .category=PET_CATEGORY_DOG};
   return 0;
}
```

The *preprocessor* deals with lines that start with `#`, such as `#include` or `#define`, and it's basically just a textual search-and-replace engine.

When the preprocessor sees `#include`, it opens the corresponding file (`stdio.h` or `pet.h`) and inlines its contents into the output.

When dealing with our `main.c`, the output of preprocessing is a long file that contains everything written in `stdio.h` (also the `#includes` inside `stdio.h` inlined here), the contents of the `pet.h` we created, and the original contents of our `main.c`.

This then goes into the compiler stage.

Note that we used double quotes for our own `#include "pet.h"` and angle brackets for `#include <stdio.h>`. The difference is where the preprocessor searches for the file. Files included with angle brackets are read from the system include directories, and files in double quotes come from the current directory where `main.c` lives.

So the whole magic above is just a complicated way to express the same thing we did last week with *students* or *coffees*. We extracted our struct and enum definition into their own header file.

This does compile right now with `gcc main.c` but doesn't do anything interesting; it just defines a `Pet` variable called `p` and exits.

We added a function to print out a pet in `main.c` first:

```
// main.c
#include <stdio.h>
#include "pet.h"

void print_pet(struct Pet p){
    printf("%s\n", p.name);
}

int main(void){
   struct Pet p = {.name="Fred", .category=PET_CATEGORY_DOG};
   print_pet(p);

   return 0;
}
```

But this function is really part of the *pet domain*, so we should move it into the `pet.c` file instead.

```
// pet.c
#include <stdio.h>
#include "pet.h"

void print_pet(struct Pet p){
    printf("%s\n", p.name);
}

// main.c
#include <stdio.h>
#include "pet.h"

int main(void){
   struct Pet p = {.name="Fred", .category=PET_CATEGORY_DOG};
```

```
    print_pet(p);
    return 0;
}
```

To compile, we give both `.c` files to gcc, but:

```
gcc main.c pet.c

main.c: In function ‘main’:
main.c:6:4: warning: implicit declaration of function ‘print_pet’ [-Wimplicit-function-declaration]
    6 |    print_pet(p);
      |    ^~~~~~~~~
```

This still errors out, because when dealing with `main.c`, the compiler doesn't know what `print_pet` is. We need to add the *function prototype*, also known as a *declaration* or *function signature*, to `pet.h` as well:

```
// pet.h
enum PetCategory {
    PET_CATEGORY_DOG,
    PET_CATEGORY_CAT,
    PET_CATEGORY_BIRD,
    PET_CATEGORY_FISH,
};

struct Pet {
    char name[33];
    enum PetCategory category;
};

// add this line over here:
void print_pet(struct Pet p);
```

Now we can really compile and run it with:

```
gcc main.c pet.c && ./a.out
Fred
```

Alternatively, you can just write `*.c`. It's the same thing, because we only have these two `.c` files in the current directory.

```
gcc *.c && ./a.out
Fred
```

The include guard idiom

The next thing we need to learn is that in big software projects, some header files can appear in many places. For example, the headers of a logging library are likely to be included multiple times. When the preprocessor prepares the input for the compiler, these headers would be included again and again, causing problems.

The simplest way to demonstrate this problem is something like:

```
#include <stdio.h>

struct Foobar{int foo; int bar;};

struct Foobar{int foo; int bar;};

int main(void) {
```

```
    return 0;
}
```

Output:

```
Compilation failed:
main.c:5:8: error: redefinition of 'Foobar'
struct Foobar{int foo; int bar;};
       ^
main.c:3:8: note: previous definition is here
struct Foobar{int foo; int bar;};
       ^
1 error generated.
```

The struct `Foobar` is defined twice, so the compiler throws an error. Of course, nobody would intentionally do this in a source file, but if `Foobar` comes from a header that is transitively included from multiple places, you can run into this situation easily. Luckily, people came up with the *include guard idiom* to solve problems like this:

```
#ifndef PET_H
#define PET_H

enum PetCategory {
    PET_CATEGORY_DOG,
    PET_CATEGORY_CAT,
    PET_CATEGORY_BIRD,
    PET_CATEGORY_FISH,
};

struct Pet {
    char name[33];
    enum PetCategory category;
};

void print_pet(struct Pet p);

#endif // PET_H
```

Basically, you add two lines to the beginning and one line to the end of each header. These are for the preprocessor.

When the preprocessor first visits `pet.h` and sees `#ifndef PET_H`, the constant `PET_H` is not defined yet, so it proceeds to the second line, where we define `PET_H` as an empty constant. Its value doesn't matter; it's just a flag for the preprocessor. Then it continues normally and inlines everything below until it reaches the `#endif`.

If for some reason `pet.h` appears multiple times in the include chain, when it encounters `#ifndef PET_H` again, `PET_H` will already be defined, so the preprocessor will skip the body entirely, and nothing will be duplicated.

It's an ugly hack that has worked for 50 years. Later, compiler developers added `#pragma once` for the same purpose. Although it's widely supported, it's not part of the C standard, so it's out of our discussion for now.

Why not put the function implementation into the header?

It's a very good question. We could try writing this, and delete `pet.c` completely:

```
// pet.h
#ifndef PET_H
```

```c
#define PET_H
enum PetCategory {
    PET_CATEGORY_DOG,
    PET_CATEGORY_CAT,
    PET_CATEGORY_BIRD,
    PET_CATEGORY_FISH,
};

struct Pet {
    char name[33];
    enum PetCategory category;
};

// maybe this should work?
#include <stdio.h>
void print_pet(struct Pet p){
    printf("%s\n", p.name);
}
#endif //PET_H
```

And in this case, `gcc main.c` would work, but only because we have just a single source file.

If you have multiple `.c` files in a large project, then each one is a separate compilation unit, meaning the compiler turns them into an object file one by one. These contain the symbols (the function names) and the compiled machine code.

The third stage is *linking*. When the *linker* runs, it tries to make one big executable like `a.out` or a library out of all the object files.

But here's the problem: the same symbol `print_pet` ends up in multiple object files. Which one is the real one? The linker has no idea that they all come from a header duplicated everywhere, so it throws an error about multiple definitions.

```
gcc *.c
duplicate symbol '_print_pet' in:
    /private/var/folders/94/939v6wdj5qs6byngxjwlc9l40000gn/T/main-faf1a7.o
    /private/var/folders/94/939v6wdj5qs6byngxjwlc9l40000gn/T/other-0e7832.o
ld: 1 duplicate symbols
clang: error: linker command failed with exit code 1 (use -v to see invocation)
```

This is why you hide the implementation in a `.c` file. You don't include `.c` files, only headers, so there won't be a *multiple definition problem*.

The linker will find the definition in exactly one of the object files and be happy about it.

What about variables/constants in the header?

We saw the same issue when we tried to introduce a constant in `pet.h`, while having `pet.c` and `main.c`

```c
// pet.h
#ifndef PET_H
#define PET_H
enum PetCategory {
    PET_CATEGORY_DOG,
    PET_CATEGORY_CAT,
    PET_CATEGORY_BIRD,
    PET_CATEGORY_FISH,
};

struct Pet {
    char name[33];
    enum PetCategory category;
};
```

```
// but constants are OK, no?
const char * const PetCategoryNames[] = {
    "dog", "cat", "bird", "fish"
};
#endif //PET_H
```

The linker hits the same problem: `PetCategoryNames` is defined multiple times, once in `main.o`, once in `pet.o`.

What about static vs extern?

Someone asked whether we can fix it by making `PetCategoryNames` static:

```
// pet.h
#ifndef PET_H
#define PET_H
enum PetCategory {
    PET_CATEGORY_DOG,
    PET_CATEGORY_CAT,
    PET_CATEGORY_BIRD,
    PET_CATEGORY_FISH,
};

struct Pet {
    char name[33];
    enum PetCategory category;
};

// but STATIC constants are OK, no?
static const char * const PetCategoryNames[] = {
    "dog", "cat", "bird", "fish"
};
#endif //PET_H
```

And the answer is… well… maybe. This would compile, because static identifiers are not externally visible in object files. The linker won't see multiple definitions.

But… the array is copied into *every* object file. Imagine you have 1000 `.c` files, each including the same header, suddenly your binary contains 1000 copies of the same array. It becomes enormous.

For small projects this might be "OK", but the correct solution is to move the definition of the array (or any constant) into a `.c` file, and keep only the declaration in the `.h` file using `extern`.

This is our final version of the pet demo:

```
// pet.h
#ifndef PET_H
#define PET_H

enum PetCategory {
    PET_CATEGORY_DOG,
    PET_CATEGORY_CAT,
    PET_CATEGORY_BIRD,
    PET_CATEGORY_FISH,
};

struct Pet {
    char name[33];
    enum PetCategory category;
};

extern const char * const PetCategoryNames[];
void print_pet(struct Pet p);

#endif // PET_H
```

```c
// pet.c
#include <stdio.h>
#include "pet.h"
const char * const PetCategoryNames[] = {
    "dog", "cat", "bird", "fish"
};

void print_pet(struct Pet p){
    printf("%s is a %s.\n", p.name, PetCategoryNames[p.category]);
}

// main.c
#include <stdio.h>
#include "pet.h"

int main(void){
    struct Pet p = {.name="Fred", .category=PET_CATEGORY_DOG};
    print_pet(p);
    return 0;
}
```

When we're talking about variables that are not constants: you need to decide whether you want a single instance for the entire project (declare it as extern in the header and define it in the .c file), or one separate instance for each module. For the latter, mark the variable as static so it stays local to that module.

It's the same story with static functions. You *can* make them `static` and keep them in the header. They will be copied into every object file, and the linker will be happy because static definitions are private to the `.o` file.

But again: that's not how it's supposed to work. You don't want the same function defined multiple times - if nothing else, it makes the binary unnecessarily large.

Put the function declaration in the `.h` file, and move the implementation into a `.c` file. You don't need to mark the prototype with `extern` because that's the default for functions.

What about types in the header?

How is it possible that we can define types in the header and nothing breaks? Because in C, types disappear in the compilation stage — the linker never sees them. It will not encounter a "multiple definition" error for the struct, because it's not in the object file at all.

Summary

This has been quite the ride, but I hope the above explanation sheds some light on what to do and why. It's a good idea to try it out on your own until you really understand what's going on.

The rule to remember is to put *structs*, *enums*, *function prototypes*, and *extern variables and extern constants* into the header. If you define *macros*, those go into the header too. Everything else — *function definitions,* global variable definitions*,* constant *definitions* belongs in the `.c` file.

If a function or variable is not used outside its `.c` file, it's good practice to mark it `static`, making it private to that module. (If you know C++, you would use an anonymous namespace for the same effect.)

Dynamic memory handling

The introductory example for dynamic memory handling is as simple as this.

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int* ptr = malloc(sizeof(int));
    if (ptr == NULL) {
        printf("memory allocation error\n");
        return 0;
    }
    *ptr = 42;

    printf("I live on the heap at %p; my value is %d.\n", (void*)ptr, *ptr);

    free(ptr);
    return 0;
}
```

Output:

```
I live on the heap at 0x600001a28040; my value is 42.
```

We need to include `stdlib.h`, which provides the functions `malloc`, `calloc`, `free`, and `realloc` used for dynamic memory handling. To use `malloc`, we must specify how much memory we need, in this case the size of an integer, and it returns a pointer of type `int*`.

This behaves like any regular pointer we have already seen. We can assign a value to what it points to and work with it exactly as if it were pointing to a local variable.

When we are done, we must call `free` to return the memory to the operating system.

Reading text

Next, we created a function that reads a word from the user, allocates just enough memory to store it on the heap, and returns the pointer.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* read_word(void){
    printf("enter word: ");
    char buf[1024];
    scanf("%1023s", buf);

    int len = strlen(buf);
    char *result = malloc(len + 1);
    if (result == NULL){
        printf("memory allocation error\n");
        return NULL;
    }
    snprintf(result, len + 1, "%s",  buf);
    return result;

}

int main(void) {
    char * word = read_word();
    if (word != NULL) {
        printf("you entered: %s\n", word);
    }
    free(word);
    return 0;
}
```

I used `snprintf` to copy the contents of the stack allocated buffer into the `result` pointer. In class, we used `strncpy`.

Notes on error handling. `malloc` returns `NULL` if something goes wrong, for example if there is not enough memory. In that case, we print an error message and return `NULL`. The `main` function must handle this. The good news is that calling `free` on `NULL` is safe, so we do not need to add special checks for that.

Reading text into a fixed length array

The next step is reading five words into an array.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* read_word(void){
    printf("enter word: ");
    char buf[1024];
    scanf("%1023s", buf);

    int len = strlen(buf);
    char *result = malloc(len + 1);
    if (result == NULL) {
        printf("memory allocation error\n");
        return NULL;
    }
    snprintf(result, len + 1, "%s",  buf);
    return result;

}


int main(void)
{
    char* words[5];

    for(int i=0; i < 5; i++) {
        words[i] = read_word();
    }

    printf("you entered: ");
    for (int i=0; i < 5; i++) {
        if (words[i] != NULL) {
            printf("%s ", words[i]);
        }
    }
    printf("\n");

    for (int i=0; i < 5; i++) {
        free(words[i]);
    }
    return 0;
}
```

We define an array of five strings (char* pointers), fill it in a loop, and print them afterwards. At the end, we must free each word individually.

Using dynamically allocated array

If we turn our *array* into a *heap allocated pointer*, we can ask the user how many words they want to enter at runtime. This allows us to get rid of the fixed value 5.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char* read_word(void){
    printf("enter word: ");
    char buf[1024];
    scanf("%1023s", buf);

    int len = strlen(buf);
    char *result = malloc(len + 1);
    if (result == NULL) {
        printf("memory allocation error\n");
        return NULL;
    }
    snprintf(result, len + 1, "%s",  buf);
    return result;

}

int main(void)
{
    int capacity;
    printf("how many words? ");
    scanf("%d", &capacity);

    char **words = malloc(capacity * sizeof(char*));
    if (words == NULL) {
        printf("memory allocation error\n");
        return 0;
    }

    for(int i=0; i < capacity; i++){
        words[i] = read_word();
    }

    printf("you entered: ");
    for(int i=0; i < capacity; i++){
        if (words[i] != NULL) {
            printf("%s ", words[i]);
        }
    }
    printf("\n");

    for(int i=0; i < capacity; i++){
        free(words[i]);
    }

    free(words);
    return 0;
}
```

We replaced the fixed length `char *words[5]` array with `char **words`. The double star can look confusing at first, but remember that a string in C is just a `char*`. So it helps to read `char **words` as “words is a pointer to strings”.

I also replaced every usage of the constant 5 with `capacity`. Since `words` now lives on the heap, we must also call `free(words)` at the end.

A typical run looks like this:

```
how many words? 3
enter word: a
enter word: v
enter word: c
you entered: a v c
```

Dynamically extending the array

But what if we do not want to ask for the number of words up front? We can start with a small array and resize it whenever it fills up. We keep reading words until the user is finished. Then we print everything and exit.

This is the last step needed to complete the program.

The first thing we need to discuss is how the user can signal when he is done. When you read something from the user with `scanf`, you are really reading from a special file like object called the input stream of your program. It works like a pipe, where your keystrokes go in at one end and `scanf` reads them out at the other.

If you press Ctrl+D on Mac or Linux, or Ctrl+Z on Windows, you tell the terminal to close the input stream. In this case `scanf` will encounter an end of file marker. When that happens it returns a special constant called `EOF`, which has the value -1. In normal cases it returns 1, meaning it could read into our buffer.

So we add a check in `read_word` that returns `NULL` if this happens.

```
char* read_word(void){
    printf("enter word: ");
    char buf[1024];
    if (scanf("%1023s", buf) != 1) {
        return NULL;
    }

    int len = strlen(buf);
    char *result = malloc(len + 1);
    if (result == NULL) {
        printf("memory allocation error\n");
        return NULL;
    }
    snprintf(result, len + 1, "%s",  buf);
    return result;

}
```

The second step is making the array grow past the initial size of 5. The third function in C's memory handling family is `realloc`, which resizes the memory region for us.

We introduce a variable `count` to track how many words have been entered. When `count` reaches `capacity`, we trigger a resize.

```
int main(void) {
    int capacity = 5;
    int count = 0;
    char **words = malloc(capacity * sizeof(char*));
    if (words == NULL) {
       printf("memory allocation error\n");
       return 0;
    }

    while(true) {
        char *word = read_word();

        if (word == NULL) {
            // error or user pressed ctrl+D
            break;
        }

        if (capacity == count) {
            char **tmp = realloc(words, (capacity + 5) * sizeof(char *));
            if(tmp == NULL){
                printf("could not extend backing array\n");
                break;
```

```
            }
            capacity += 5;
            words = tmp;
        }

        words[count] = word;
        count++;
    }

    printf("\n");
    printf("you entered: ");
    for(int i=0;i<count;i++){
         printf("%s ", words[i]);
    }
    printf("\n");

    for(int i=0;i<count;i++){
        free(words[i]);
    }
    free(words);
    return 0;
}
```

When `realloc` succeeds, it returns a pointer to the newly allocated memory area, which we store in `tmp`. The text entered by the user is copied to this new memory location. Our `words` variable still points to the original location, but that location has been freed by `realloc`.

However, if `realloc` fails for some reason, the return pointer becomes `NULL`, and our `words` variable is still valid because no copying has happened.

We need to handle this error case and exit the loop after printing an error message. In the normal case, we simply set `words` to the new location stored in `tmp` and increase the capacity by 5.

The final program looks like this.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>

char* read_word(void){
    printf("enter word: ");
    char buf[1024];
    if (scanf("%1023s", buf) != 1) {
        return NULL;
    }

    int len = strlen(buf);
    char *result = malloc(len + 1);
    if (result == NULL) {
        printf("memory allocation error\n");
        return NULL;
    }
    snprintf(result, len + 1, "%s",  buf);
    return result;

}

int main(void) {
    int capacity = 5;
    int count = 0;
    char **words = malloc(capacity * sizeof(char*));
    if (words == NULL) {
       printf("memory allocation error\n");
       return 0;
    }

    while(true) {
        char *word = read_word();
```

```
        if (word == NULL) {
            break;
        }

        if (capacity == count) {
            char **tmp = realloc(words, (capacity + 5) * sizeof(char *));
            if(tmp == NULL){
                printf("could not extend backing array\n");
                free(word); // <- Oops, I forgot this in class
                break;
            }
            capacity += 5;
            words = tmp;
        }

        words[count] = word;
        count++;
    }

    printf("\n");
    printf("you entered: ");
    for(int i=0;i<count;i++){
         printf("%s ", words[i]);
    }
    printf("\n");

    for(int i=0;i<count;i++){
        free(words[i]);
    }
    free(words);
    return 0;
}
```

What not to do

Dynamic memory handling is the hardest part of C. This is mostly because the language has no built in notion of ownership and doesn't know if a pointer refers to a valid memory area or not. You must follow best practices such as ownership transfer or reference counting, and none of these can be expressed in the type system. C is an old language, and many newer languages like C++, Rust, Java, C#, or Python have answers to this question this way or other.

There are general rules for things you must never do with pointers:

- *dereferencing NULL*,
- *dereferencing an invalid pointer*,
- *out of bounds read and write*.

We covered these earlier when we talked about pointers. With dynamically allocated memory, the list becomes even longer:

- freeing up a pointer more than once,
- forgetting to free a pointer,
- freeing memory that was never allocated dynamically or
- using a pointer after it has been freed.

Any of these can cause memory leaks, crashes, or security vulnerabilities such as *denial of service attacks*, *heap overflows*, or local / remote code execution.

The compiler cannot catch these mistakes, but GCC provides some command line switches that help detect problems at runtime.

- `-g` with `-fno-omit-frame-pointer` gives better error messages upon crashes with accurate line numbers.
- `-fsanitize=address,undefined` enables *runtime sanitizers* for memory and undefined behavior bugs.

The *address* sanitizer helps catch memory errors. The *undefined behavior* sanitizer detects issues such as division by zero or integer overflow.

I recommend compiling with this full set of flags from now on:

```
gcc -g -fsanitize=address,undefined -fno-omit-frame-pointer -W -Wall -Wextra -pedantic -Werror myfil
e.c
```

If you want to give a better name to the executable (not `a.out`), you can also use `-o my-program` or whatever name you choose. So a complete example looks like:

```
gcc -g -fsanitize=address,undefined -fno-omit-frame-pointer -W -Wall -Wextra -pedantic -Werror myfil
e.c -o my-program
```

We built this step by step over the weeks, so hopefully it is not too overwhelming.

Here is a sample program that demonstrates what not to do and how the sanitizer helps. *I do not know how well this works on Windows*, but the address sanitizer *works on Linux* and on my *Mac* as well.

```c
#include <stdlib.h>
#include <stdio.h>
int main(void) {

    // double free
    char *p = malloc(10);
    free(p);
    free(p);

    // invalid free:
    int x = 42;
    free(&x);

    // use-after-free (undefined behavior!)
    char *p2 = malloc(10);
    p2[0] = 'A';       // OK
    p2[1] = 'B';       // OK
    free(p2);          // memory is now invalid
    p2[0] = 'X';

    // memory leak
    char *p3 = malloc(10);

    //undefined behavior
    printf("%d", 1/0);

    return 0;
}
```

When compiled, this program produces many warnings. When run with sanitizers turned on, the output looks something like this:

```
=================================================================
==73198==ERROR: AddressSanitizer: attempting double-free on 0x0001072006f0 in thread T0:
    #0 0x104e33260 in wrap_free+0x98 (libclang_rt.asan_osx_dynamic.dylib:arm64e+0x53260)
    #1 0x10458b7f0 in main week7.c:8
    #2 0x18cc890dc  (<unknown module>)

0x0001072006f0 is located 0 bytes inside of 10-byte region [0x0001072006f0,0x0001072006fa)
freed by thread T0 here:
    #0 0x104e33260 in wrap_free+0x98 (libclang_rt.asan_osx_dynamic.dylib:arm64e+0x53260)
```

```
    #1 0x10458b7e8 in main week7.c:7
    #2 0x18cc890dc  (<unknown module>)

previously allocated by thread T0 here:
    #0 0x104e33124 in wrap_malloc+0x94 (libclang_rt.asan_osx_dynamic.dylib:arm64e+0x53124)
    #1 0x10458b7dc in main week7.c:6
    #2 0x18cc890dc  (<unknown module>)

SUMMARY: AddressSanitizer: double-free (libclang_rt.asan_osx_dynamic.dylib:arm64e+0x53260) in wrap_f
ree+0x98
==73198==ABORTING
[1]    73198 abort      ./main
```

`AddressSanitizer` catches the double free error and shows that the problem is in `week7.c` line 8, so we know exactly where to look.

This is still not a perfect safety net, because we are talking about C. Sanitizers catch many, but not all, bugs; undefined behavior can still slip through. It will not catch everything, but it can prevent many problems and save you time during the exam. It is definitely worth trying and learning to use.

Homework

While working on the problems below, use the sanitizer to detect your mistakes, learn to read its output.

Practice header files

Go back to the previous week and make sure you can implement the structs mentioned there. Move your implementation to header files (like in our `pet` example today), and call them from `main.c`.

Sort Integers by Parity

Write a function that takes an array of integers and returns a new dynamically allocated array where even numbers come first and odd numbers follow. Don't forget to free it up at the end.

Reverse a string

Write a function that takes a string parameter as `const char*`, and returns a dynamically allocated string which contains the characters of the parameter backwards. Read the string from the user, print it out reversed. Free up the memory!

A heap of pets

Use one of the structs from earlier, say the `pet` example. Allocate memory that can store five pets on the `heap`, fill it up with some values and print them out. Don't forget to free up everything.

Dynamically growing list

Using the final form of the `words` example from above, make a program that reads pets from the user, and adds them to a growing array.

Print out the pets at the end, and clean up the resources.